Introduction to Flagger

— A Progressive Delivery Operator for Kubernetes —

@mathetake — Kubernete Meetup Tokyo #32

whoami

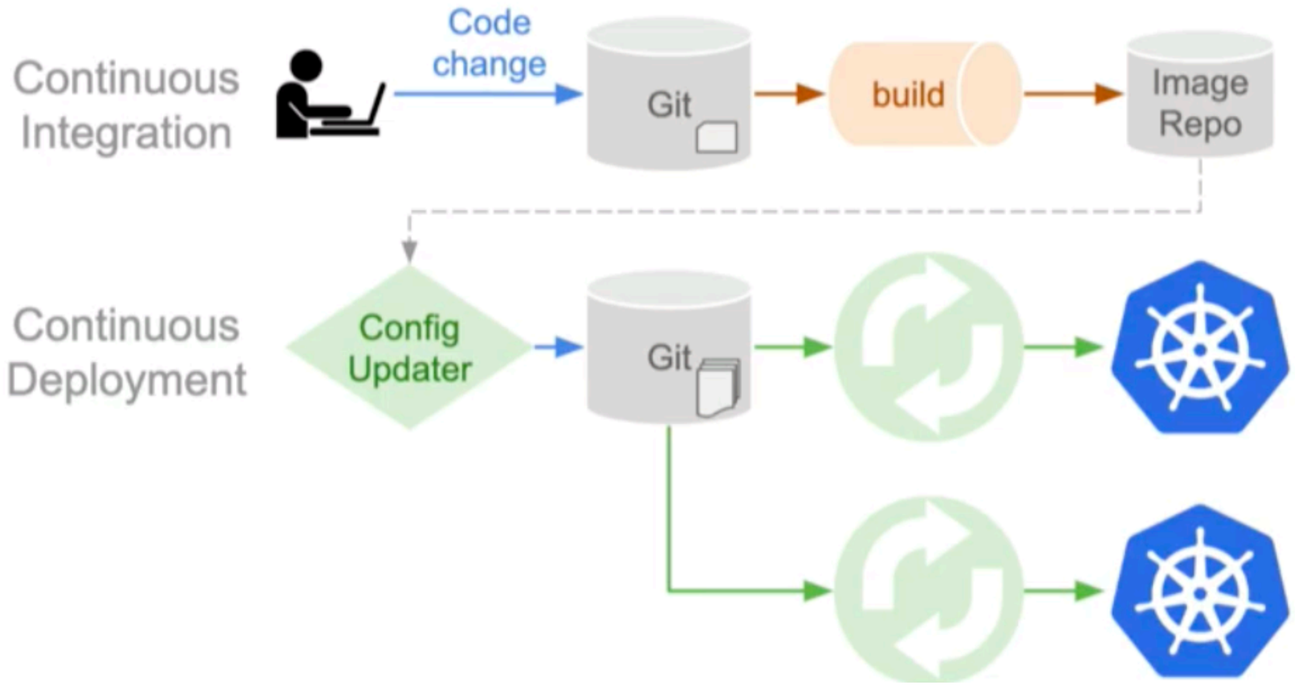
- Takeshi Yoneda (マスタケ, @mathetake)
  - DMM.com, Data Application Engineering, Team leader
  - website: https://mathetake.github.io/
  - Twitter: https://twitter.com/mathetake
  - Github: https://github.com/mathetake
- Maintainer of weaveworks/flagger https://docs.flagger.app/
- Member of proxy-wasm : https://github.com/proxy-wasm
- 最近Envoyをビルドするためだけに Ryzen 9 3950xでPC組みました

1. Background

※gitops前提で話しますが, 別の宗派でも共通する話です

Continuous X

- みんな大好きContinuous X
  - Cloud Native界隈のスタンダード
  - ないと生きていけない
  - 豊富なsoftware群
    - Spinnaker, Flux, ArgoCD, …
- Goal: Agility & Reliability
- Automated testing, deployment



https://www.weave.works/blog/automate-kubernetes-with-gitops

Continuous X

- Continuous X allows us to test/deploy tons of applications in a day
  - PRマージでデプロイ完了、手オペ無し
- Rollback = PRのリバート
  - 簡単!
- Reconciliation Loop万歳🙌
- 怖いものはない

Continuous X

- Continuous X allows us to test/deploy tons of applications in a day
  - PRマージでデプロイ完了、手オペ無し
- Rollback = PRのリバート
  - 簡単!
- Reconciliation Loop万歳🙌
- ~~怖いものはない~~

No Free Lunch

Continuous Delivery is hard

- CDによるdeploy後のreliability…?
  - deploy直後はログ/メトリクスを見ざるを得ない
    - やばかったらリバート！
- 結局Human in the loop
  - ロールバックのオペレーションで事故る可能性
  - We humans are all fallible

Continuous Delivery is hard

- CDによるdeploy後のreliability…?
  - deploy直後はログ/メトリクスを見ざるを得ない
    - やばかったらリバート！
- 結局Human in the loop
  - ロールバックのオペレーションで事故る可能性
  - We humans are all fallible

もっといい感じにシュッとしたい

What is Progressive Delivery

Progressive Delivery is a term that includes deployment strategies that try to avoid the pitfalls of all-or-nothing deployment strategies

https://carlossg.github.io/presentations/2019-06_cdsummit/#/2/1

What is Progressive Delivery

Progressive Delivery is a term that includes deployment strategies that try to avoid the pitfalls of all-or-nothing deployment strategies

https://carlossg.github.io/presentations/2019-06_cdsummit/#/2/1

What is Progressive Delivery

- NaiveなCDだとAll-or-nothing deployment
  - 新しいversionで古いやつをすべて入れ替える
  - ロールバックはまたその逆
- ***Progressive Delivery = CD++**
  - 普通のCDをよりシュッとするもの
  - All-or-nothingではなく、徐々にデプロイ
    - 「徐々にデプロイ」を自動化
    - **本番デプロイのリスクを減らすもの**

* https://qiita.com/mumoshu/items/63b29bca6a052d8c7087

Test in production meets P.D.

- Progressive Delivery = 徐々に本番にデプロイしていく

  ≒ 本番環境で新しいバージョンを試す(test)

- ‘Test in production’ の考え方に近い
  - Staging cannot be production
  - iterationの高速化, Agilityの向上

💀 All-or-nothing deployment + Test in production = high risk

🍺 **Progressive Delivery + Test in production = low risk**

https://www.getambassador.io/docs/latest/topics/concepts/progressive-delivery/

Implementation

- Spinnaker
- K8s native
  - Argo-rollouts
    - nativeなservice based*
  - **weaveworks/flagger**
    - Service Mesh/Ingress Controllerベース
- 基本動作: 少しrollout ⇔ メトリクスのチェック

*昔のバージョンはservice meshと合わせてtraffic shiftingができなかった

2. Introduction to Flagger

What is Flagger

- Progressive DeliveryのためのOSS
  - github.com/weaveworks/flagger
  - weaveworks社のStefanが開発
- 2020年6月17日にGA
- Istioを前提としたものとして開発

  → その後複数platformをサポート

Give developers confidence in automating the production releases

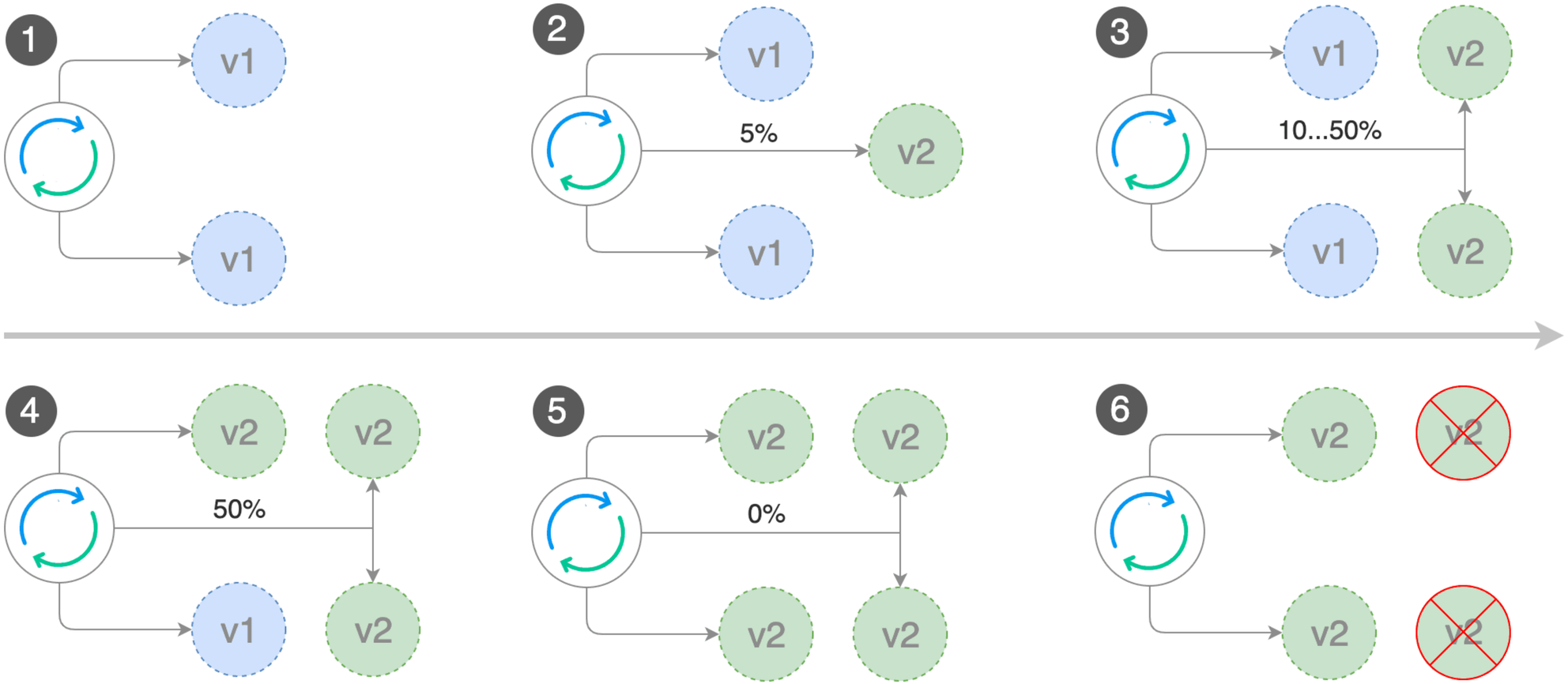
Flagger’s Features

- Service mesh native
  - Istio, SMI(linkerd, crossover), Appmesh + ingress controllers
  - Fine tuned traffic shifting
- Gitops native
- multiple deployment strategies
- Custom metrics
- Manual gating(approve, pause, resume), Webhooks
- Alerting

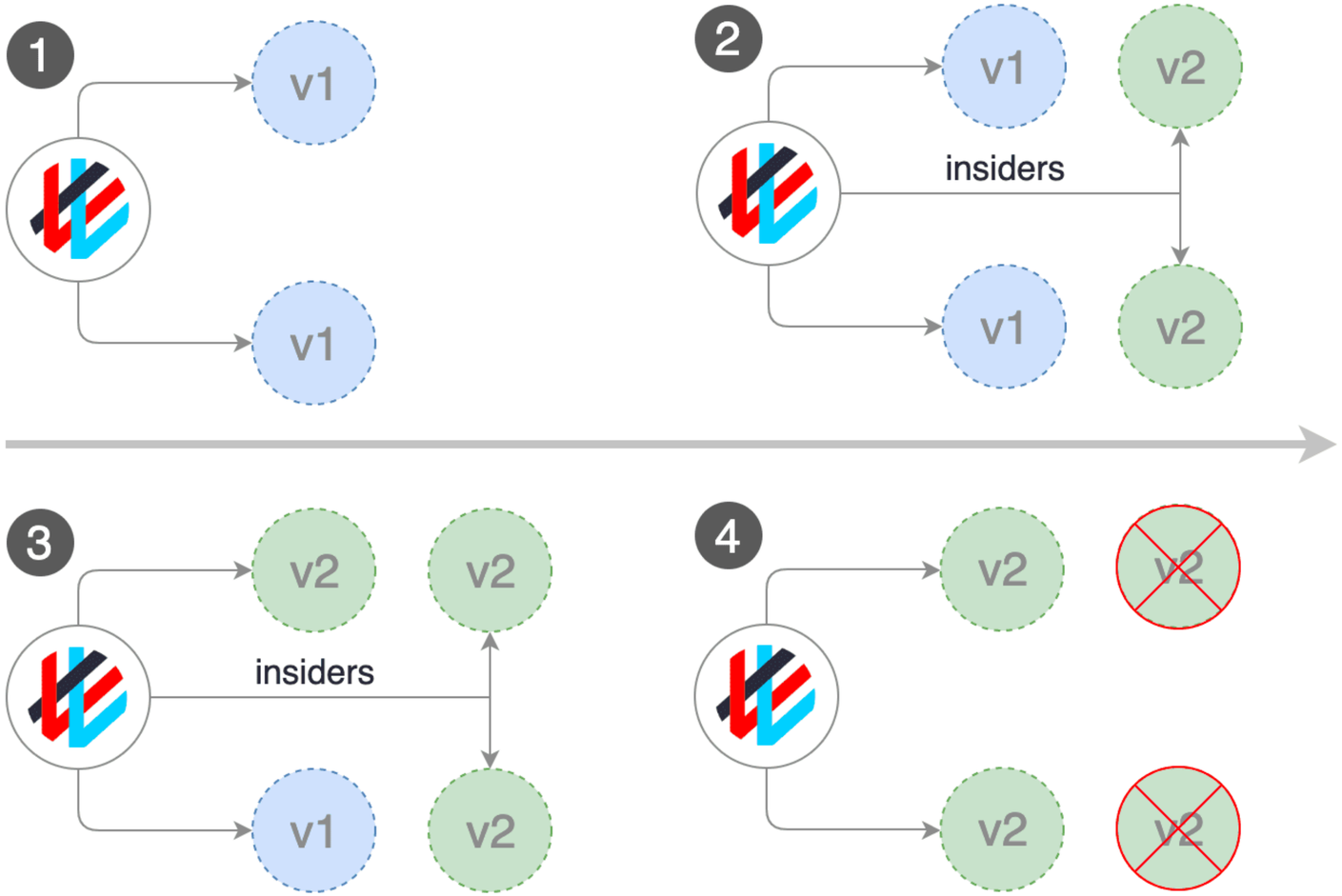
Deployment Strategy

1. **Canary Release (progressive traffic shifting)**
   - http, grpcなアプリケーション用
2. **A/B Testing (HTTP headers and cookies traffic routing)**
   - Session affinityが必要なアプリケーション用
3. **Blue/Green (traffic switching)**
   - Any workload
4. **Blue/Green Mirroring (traffic shadowing)**
   - 冪等なアプリケーション用(最も安全)
   - Canary Releaseの前段階としてmirroringすることも可能

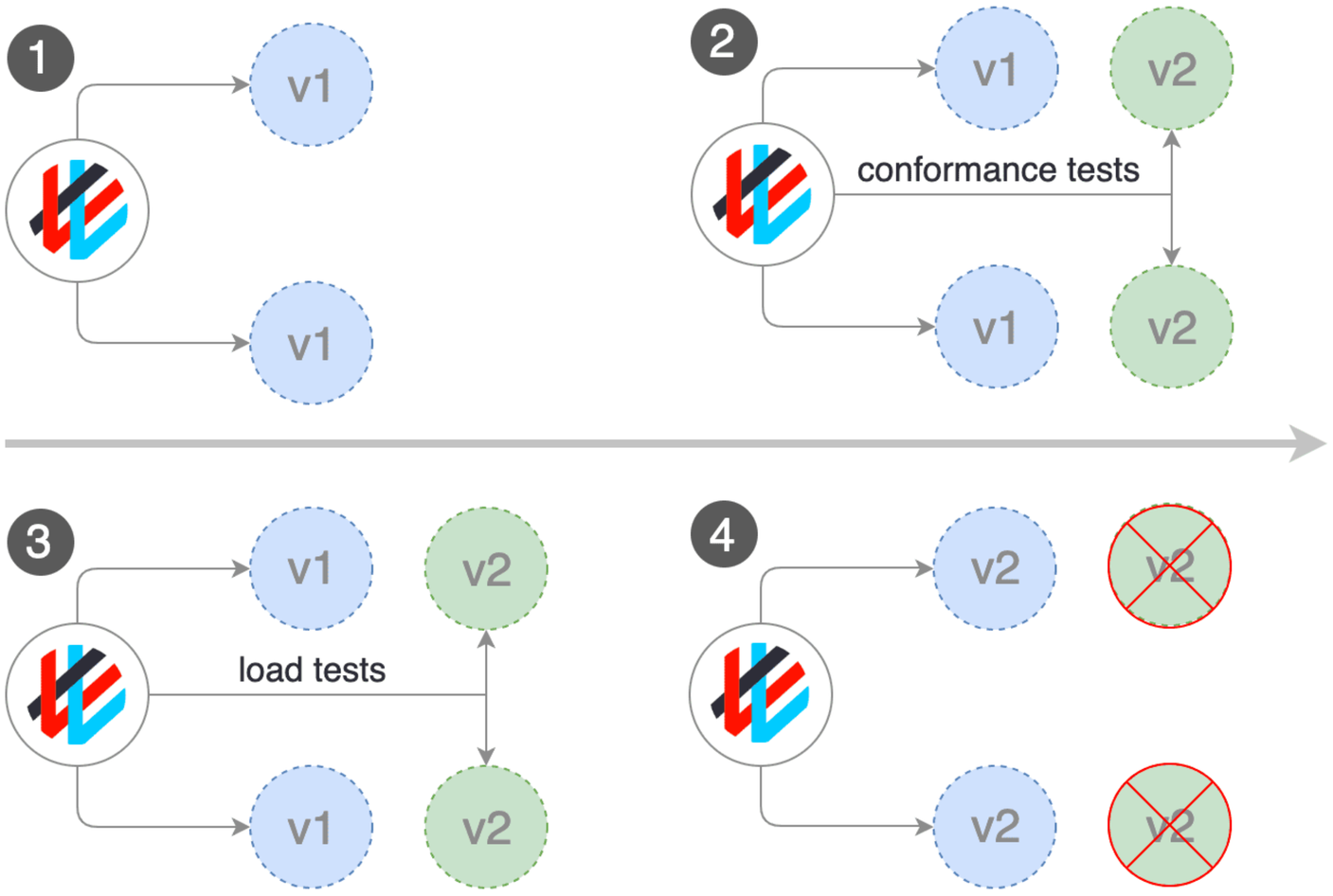
Deployment Strategy - Canary Release



Deployment Strategy - A/B testing

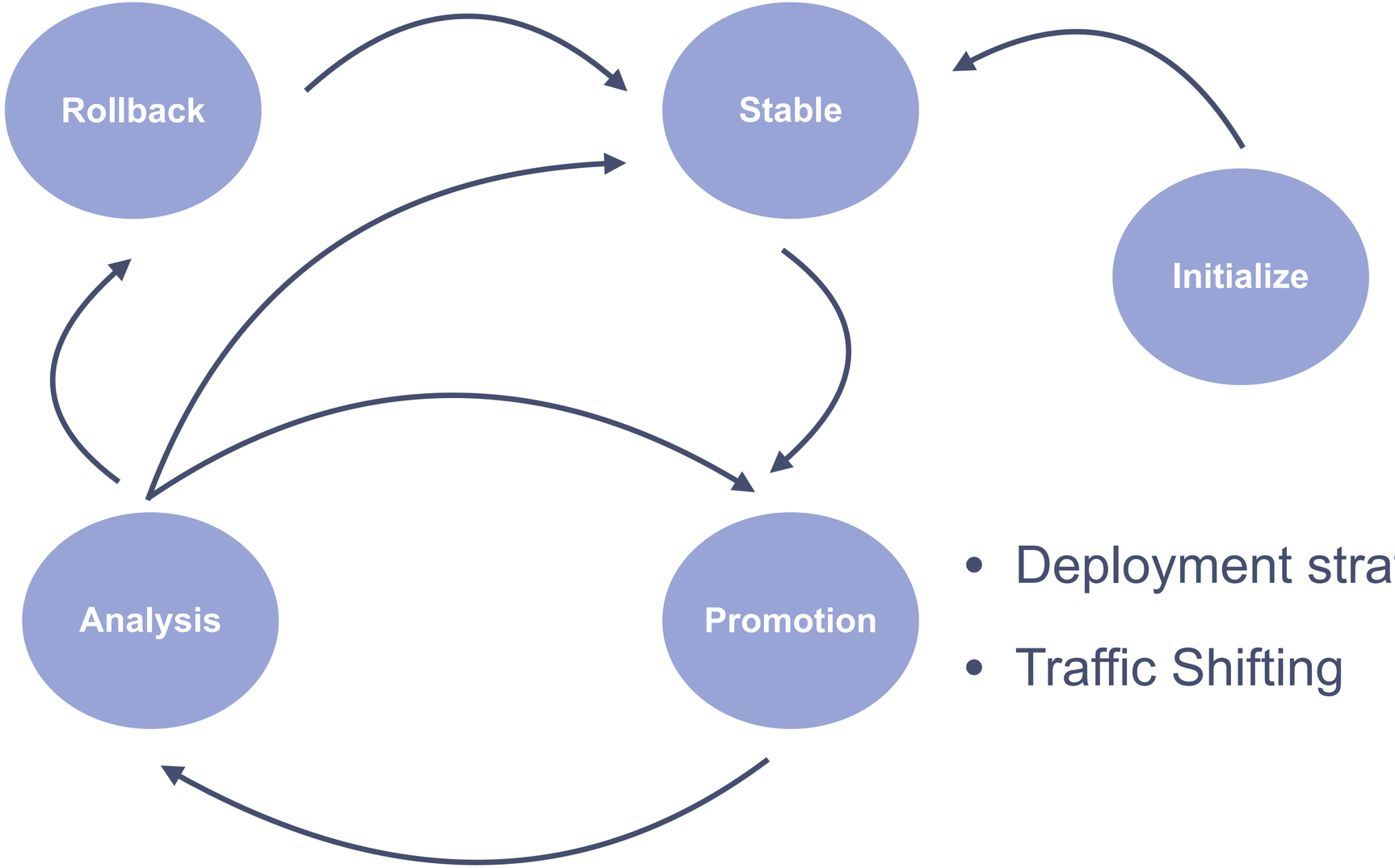


Deployment Strategy - Blue Green



Control loop / phase

- Alert provider



- webhook
- metrics check
- Manual gating

- Deployment strategy
- Traffic Shifting

Flagger CRD

1. **Canary**
   - メインのリソース
   - 最低限の機能であればこれだけでOK
2. **MetricTemplate**
   - Analyzeするmetricsのクエリ
   - Metric プロバイダーの指定
3. **Alertprovider**
   - deliveryの通知先を指定

Flagger CRD: Canary

```yaml
apiVersion: flagger.app/v1beta1
kind: Canary
metadata:
  name: podinfo
  namespace: test
spec:
  provider: istio
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: podinfo
  progressDeadlineSeconds: 60
  autoscalerRef:
    apiVersion: autoscaling/v2beta1
    kind: HorizontalPodAutoscaler
    name: podinfo
  service:
    name: podinfo
    port: 9898
    targetPort: 9898
    portName: http
    portDiscovery: true
    match:
      - uri:
          prefix: /
    timeout: 5s
```

```yaml
analysis:
  interval: 1m
  threshold: 10
  maxWeight: 50
  stepWeight: 5
  metrics:
  - name: request-success-rate
    thresholdRange:
      min: 99
    interval: 1m
  - name: "database connections"
    templateRef:
      name: db-connections
    thresholdRange:
      min: 2
      max: 100
    interval: 1m
  webhooks:
    - name: "load test"
      type: rollout
      url: http://flagger-loadtester.test/
      metadata:
        cmd: “…”
```

Flagger CRD: Canary

Applicationの指定

- Deployment
- Daemonset

```yaml
apiVersion: flagger.app/v1beta1
kind: Canary
metadata:
  name: podinfo
  namespace: test
spec:
  provider: istio
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: podinfo
  progressDeadlineSeconds: 60
  autoscalerRef:
    apiVersion: autoscaling/v2beta1
    kind: HorizontalPodAutoscaler
    name: podinfo
  service:
    name: podinfo
    port: 9898
    targetPort: 9898
    portName: http
    portDiscovery: true
    match:
      - uri:
          prefix: /
    timeout: 5s
```

```yaml
analysis:
  interval: 1m
  threshold: 10
  maxWeight: 50
  stepWeight: 5
  metrics:
  - name: request-success-rate
    thresholdRange:
      min: 99
    interval: 1m
  - name: "database connections"
    templateRef:
      name: db-connections
    thresholdRange:
      min: 2
      max: 100
    interval: 1m
  webhooks:
    - name: "load test"
      type: rollout
      url: http://flagger-loadtester.test/
      metadata:
        cmd: “…”
```

Flagger CRD: Canary

HPAの指定(optional)

```yaml
apiVersion: flagger.app/v1beta1
kind: Canary
metadata:
  name: podinfo
  namespace: test
spec:
  provider: istio
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: podinfo
  progressDeadlineSeconds: 60
  autoscalerRef:
    apiVersion: autoscaling/v2beta1
    kind: HorizontalPodAutoscaler
    name: podinfo
  service:
    name: podinfo
    port: 9898
    targetPort: 9898
    portName: http
    portDiscovery: true
    match:
      - uri:
          prefix: /
    timeout: 5s
```

```yaml
analysis:
  interval: 1m
  threshold: 10
  maxWeight: 50
  stepWeight: 5
  metrics:
  - name: request-success-rate
    thresholdRange:
      min: 99
    interval: 1m
  - name: "database connections"
    templateRef:
      name: db-connections
    thresholdRange:
      min: 2
      max: 100
    interval: 1m
  webhooks:
    - name: "load test"
      type: rollout
      url: http://flagger-loadtester.test/
      metadata:
        cmd: “…”
```

Flagger CRD: Canary

```yaml
apiVersion: flagger.app/v1beta1
kind: Canary
metadata:
  name: podinfo
  namespace: test
spec:
  provider: istio
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: podinfo
  progressDeadlineSeconds: 60
  autoscalerRef:
    apiVersion: autoscaling/v2beta1
    kind: HorizontalPodAutoscaler
    name: podinfo
  service:
    name: podinfo
    port: 9898
    targetPort: 9898
    portName: http
    portDiscovery: true
    match:
      - uri:
          prefix: /
    timeout: 5s
```

serviceの定義

```yaml
analysis:
  interval: 1m
  threshold: 10
  maxWeight: 50
  stepWeight: 5
  metrics:
  - name: request-success-rate
    thresholdRange:
      min: 99
    interval: 1m
  - name: "database connections"
    templateRef:
      name: db-connections
    thresholdRange:
      min: 2
      max: 100
    interval: 1m
  webhooks:
    - name: "load test"
      type: rollout
      url: http://flagger-loadtester.test/
      metadata:
        cmd: “…”
```

Flagger CRD: Canary

```yaml
apiVersion: flagger.app/v1beta1
kind: Canary
metadata:
  name: podinfo
  namespace: test
spec:
  provider: istio
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: podinfo
  progressDeadlineSeconds: 60
  autoscalerRef:
    apiVersion: autoscaling/v2beta1
    kind: HorizontalPodAutoscaler
    name: podinfo
  service:
    name: podinfo
    port: 9898
    targetPort: 9898
    portName: http
    portDiscovery: true
    match:
      - uri:
          prefix: /
    timeout: 5s
```

```yaml
analysis:
  interval: 1m
  threshold: 10
  maxWeight: 50
  stepWeight: 5
  metrics:
  - name: request-success-rate
    thresholdRange:
      min: 99
    interval: 1m
  - name: "database connections"
    templateRef:
      name: db-connections
    thresholdRange:
      min: 2
      max: 100
    interval: 1m
  webhooks:
    - name: "load test"
      type: rollout
      url: http://flagger-loadtester.test/
      metadata:
        cmd: “…”
```

Canary analysisの定義

Flagger CRD: MetricTemplate

```yaml
apiVersion: flagger.app/v1beta1
kind: MetricTemplate
metadata:
  name: not-found-percentage
  namespace: istio-system
spec:
  provider:
    type: prometheus
    address: http://promethues.istio-system:9090
  query: |
    100 - sum(
        rate(
            istio_requests_total{
              reporter="destination",
              destination_workload_namespace="{{ namespace }}",
              destination_workload="{{ target }}",
              response_code!="404"
            }[{{ interval }}]
        )
    )
    /
    sum(
        rate(
            istio_requests_total{
              reporter="destination",
              destination_workload_namespace="{{ namespace }}",
              destination_workload="{{ target }}"
            }[{{ interval }}]
        )
    ) * 100
```

Flagger CRD: MetricTemplate

Metricプロバイダの指定

- Prometheus
- Datadog
- CloudWatch

```yaml
apiVersion: flagger.app/v1beta1
kind: MetricTemplate
metadata:
  name: not-found-percentage
  namespace: istio-system
spec:
  provider:
    type: prometheus
    address: http://promethues.istio-system:9090
  query: |
    100 - sum(
        rate(
            istio_requests_total{
              reporter="destination",
              destination_workload_namespace="{{ namespace }}",
              destination_workload="{{ target }}",
              response_code!="404"
            }[{{ interval }}]
        )
    )
    /
    sum(
        rate(
            istio_requests_total{
              reporter="destination",
              destination_workload_namespace="{{ namespace }}",
              destination_workload="{{ target }}"
            }[{{ interval }}]
        )
    ) * 100
```

Flagger CRD: MetricTemplate

クエリのtemplate

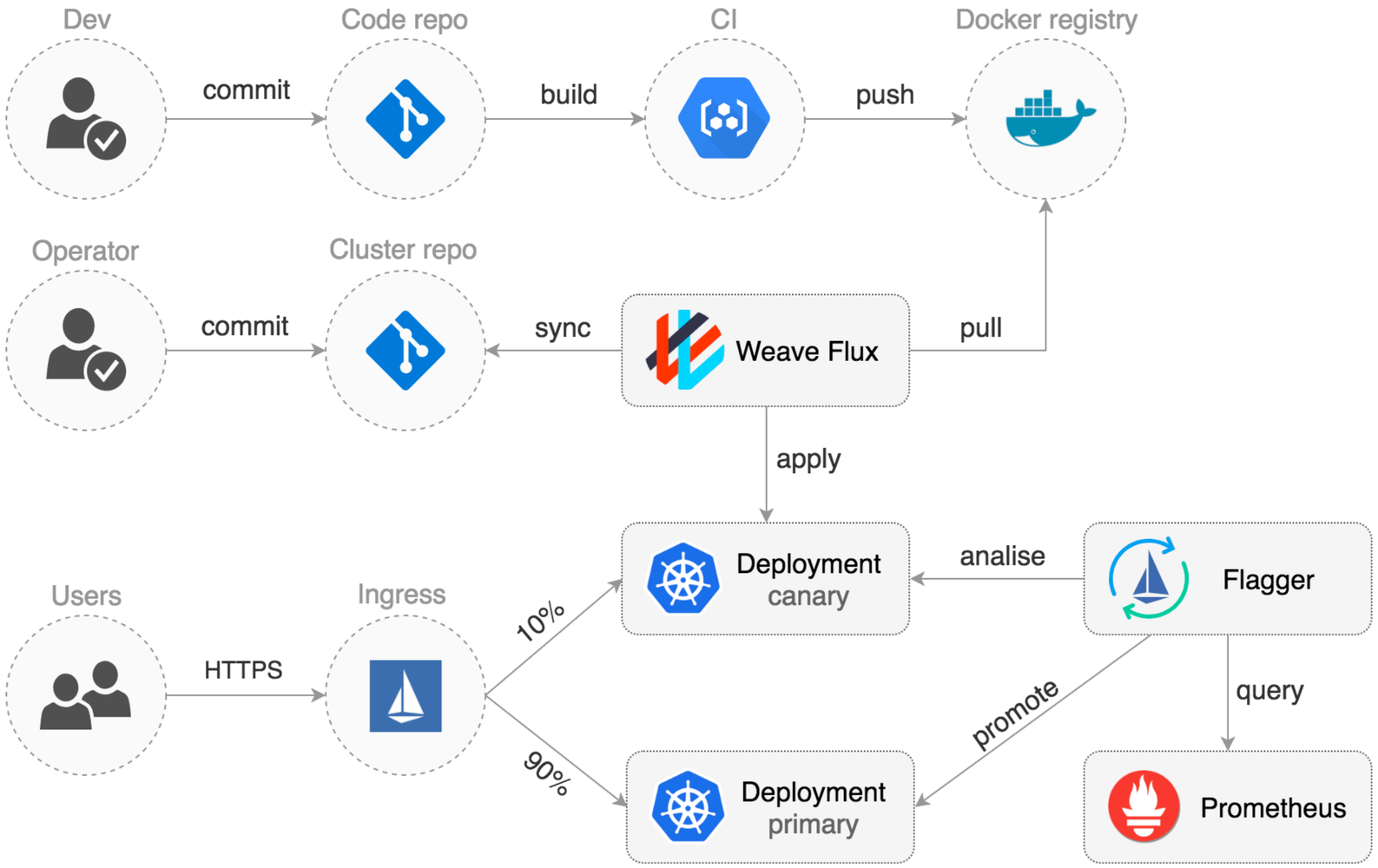
```yaml
apiVersion: flagger.app/v1beta1
kind: MetricTemplate
metadata:
  name: not-found-percentage
  namespace: istio-system
spec:
  provider:
    type: prometheus
    address: http://promethues.istio-system:9090
  query: |
    100 - sum(
        rate(
            istio_requests_total{
              reporter="destination",
              destination_workload_namespace="{{ namespace }}",
              destination_workload="{{ target }}",
              response_code!="404"
            }[{{ interval }}]
        )
    )
    /
    sum(
        rate(
            istio_requests_total{
              reporter="destination",
              destination_workload_namespace="{{ namespace }}",
              destination_workload="{{ target }}"
            }[{{ interval }}]
        )
    ) * 100
```

Flagger CRD: Alert Provider

```yaml
apiVersion: flagger.app/v1beta1
kind: AlertProvider
metadata:
  name: on-call
  namespace: flagger
spec:
  type: slack
  channel: on-call-alerts
  username: flagger
  # address: https://hooks.slack.com/services/YOUR/SLACK/WEBHOOK
  secretRef:
    name: on-call-url
---
apiVersion: v1
kind: Secret
metadata:
  name: on-call-url
  namespace: flagger
data:
  address: <encoded-url>
```

3. How Flagger works?

Overview



https://github.com/stefanprodan/gitops-progressive-delivery

Canary Initialization

- **Canary CRを作成した時点でFlaggerは複数のリソースを作る**
- アプリケーション関係: 基本的に元の定義のコピー
  - Deployment/Daemonset: <canary.spec.targetRef.name>-primary
    - “app: foo” → “app: foo-primary” とラベルを変換
  - HPA: <canary.spec.autoscalerRef.name>-primary
  - mountしてるconfigmap, secret: <name>-primary

Canary Initialization

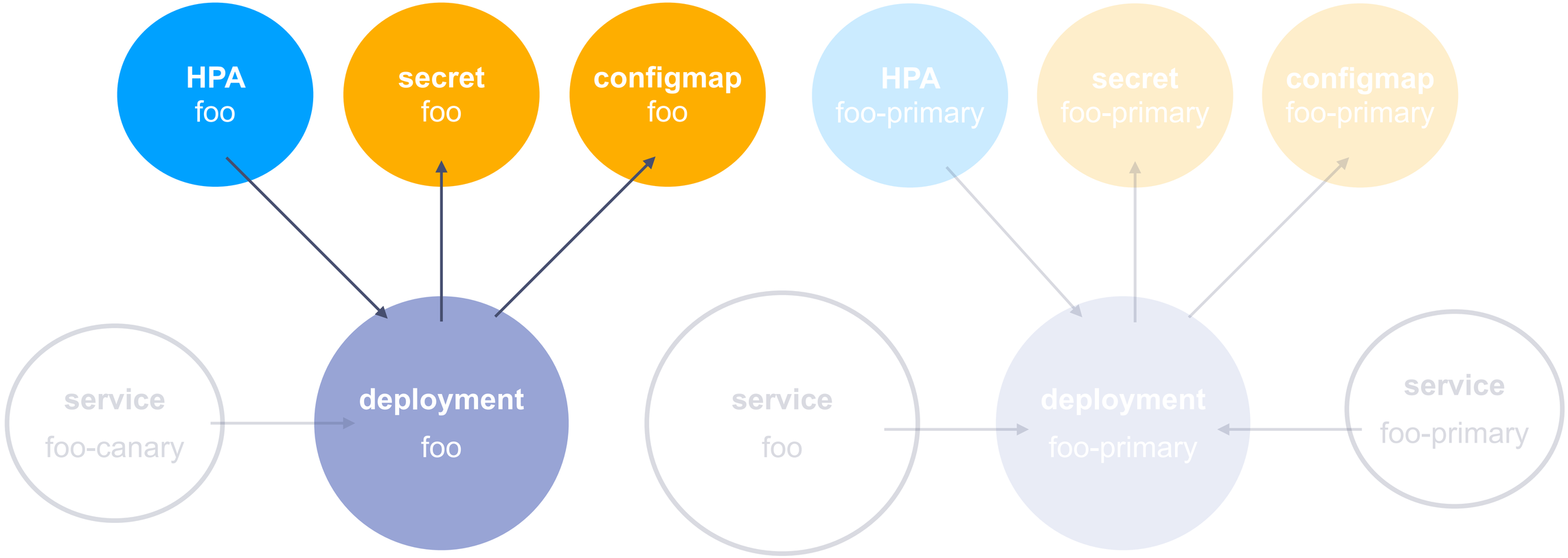
- **Canary CRを作成した時点でFlaggerは複数のリソースを作る**
- アプリケーション関係: 基本的に元の定義のコピー
  - Deployment/Daemonset: <canary.spec.targetRef.name>-primary
    - “app: foo” → “app: foo-primary” とラベルを変換
  - HPA: <canary.spec.autoscalerRef.name>-primary
  - mountしてるconfigmap, secret: <name>-primary

既存のdeployment,HPAをそのまま使える

Canary Initialization

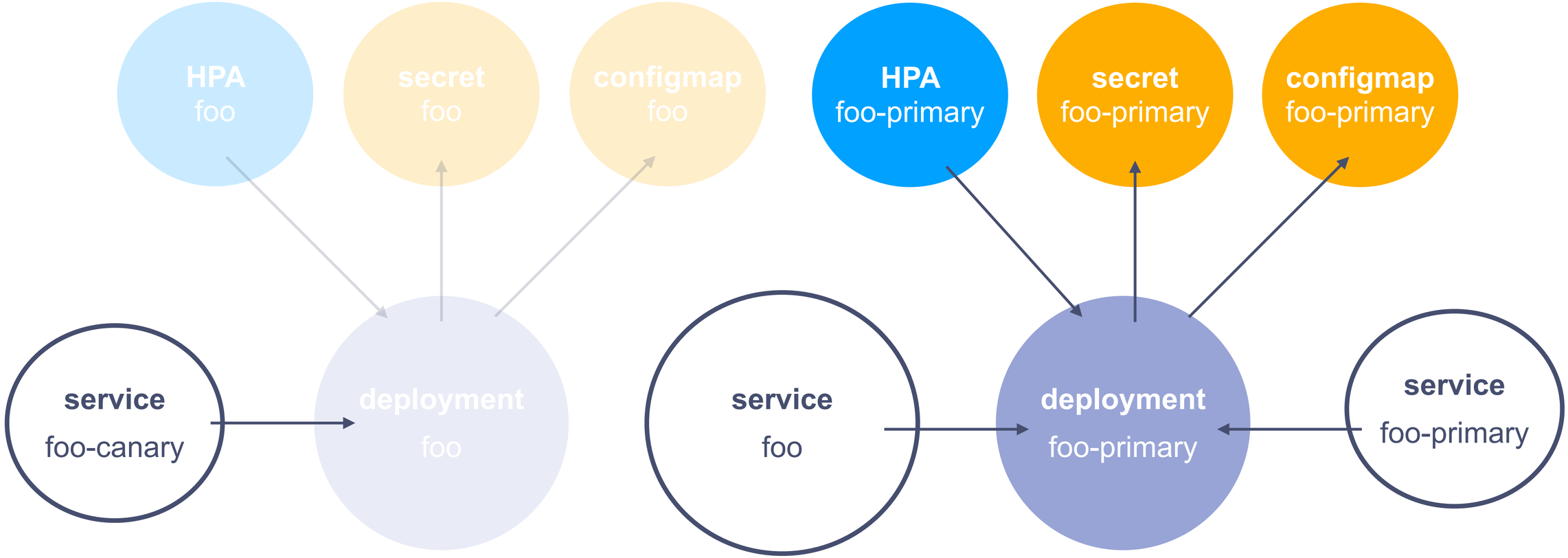
- 元のuser managedなdeploymentを **canary** と呼ぶ
- コピーされたFlagger managedなdeploymentを **primary** と呼ぶ
- 諸々の作成が完了後、user managedなdeploymentをreplicas = 0にする
  - (replicasは元々設定されていない前提)
  - Canaryのpodはゼロ個になる

Canary Initialization



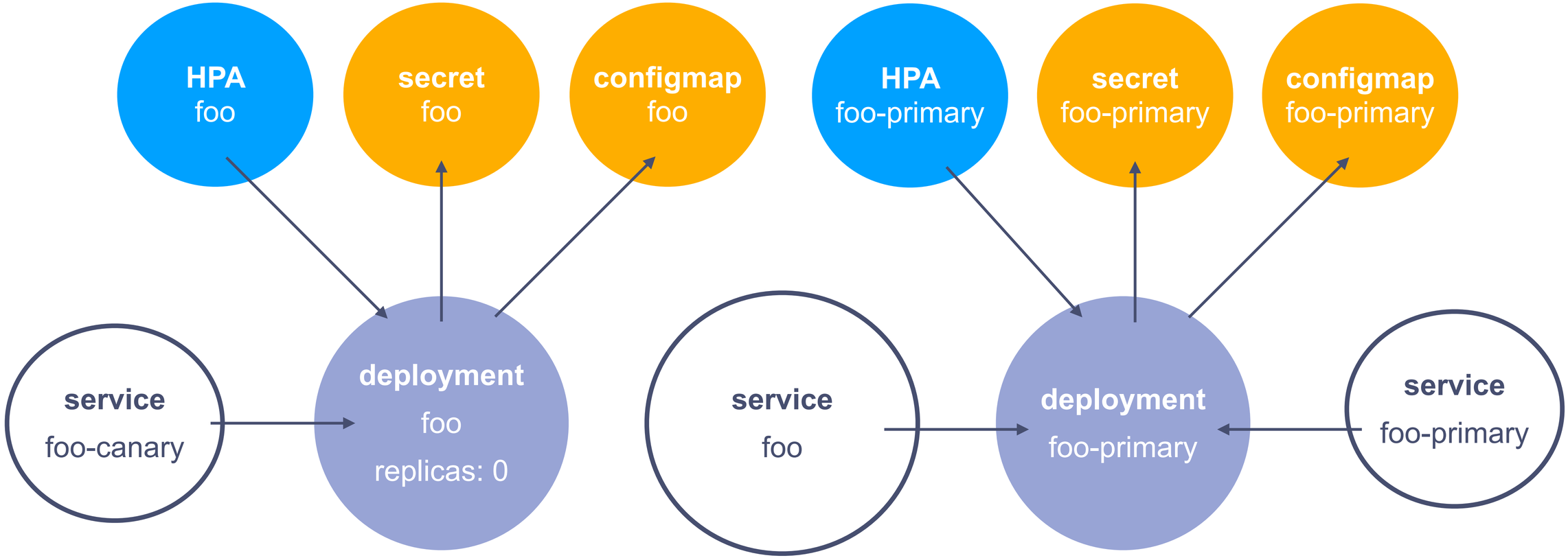
Managed by Users = Canary

Canary Initialization



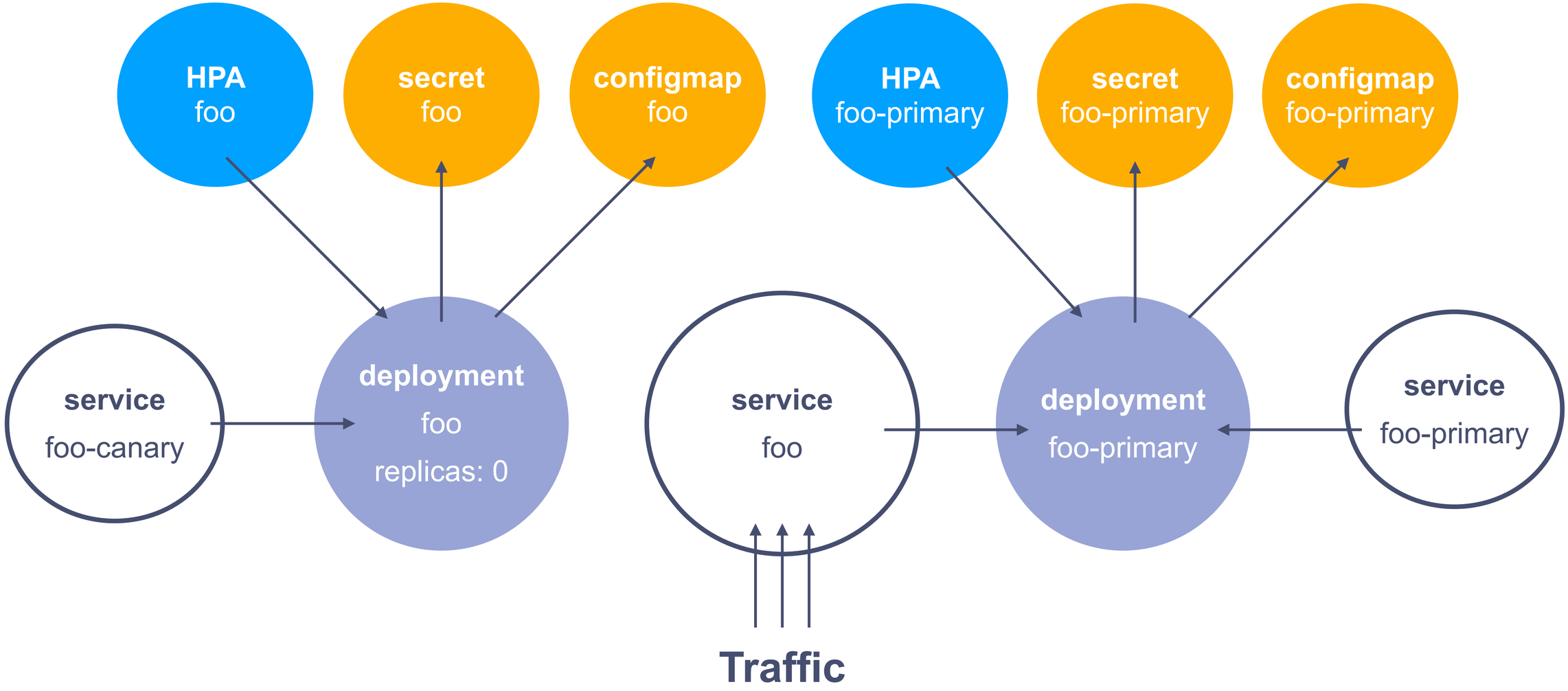
Managed by Flagger = Primary

Canary Initialization



Set replicas = 0 on canary

Canary Initialization



Change Detection

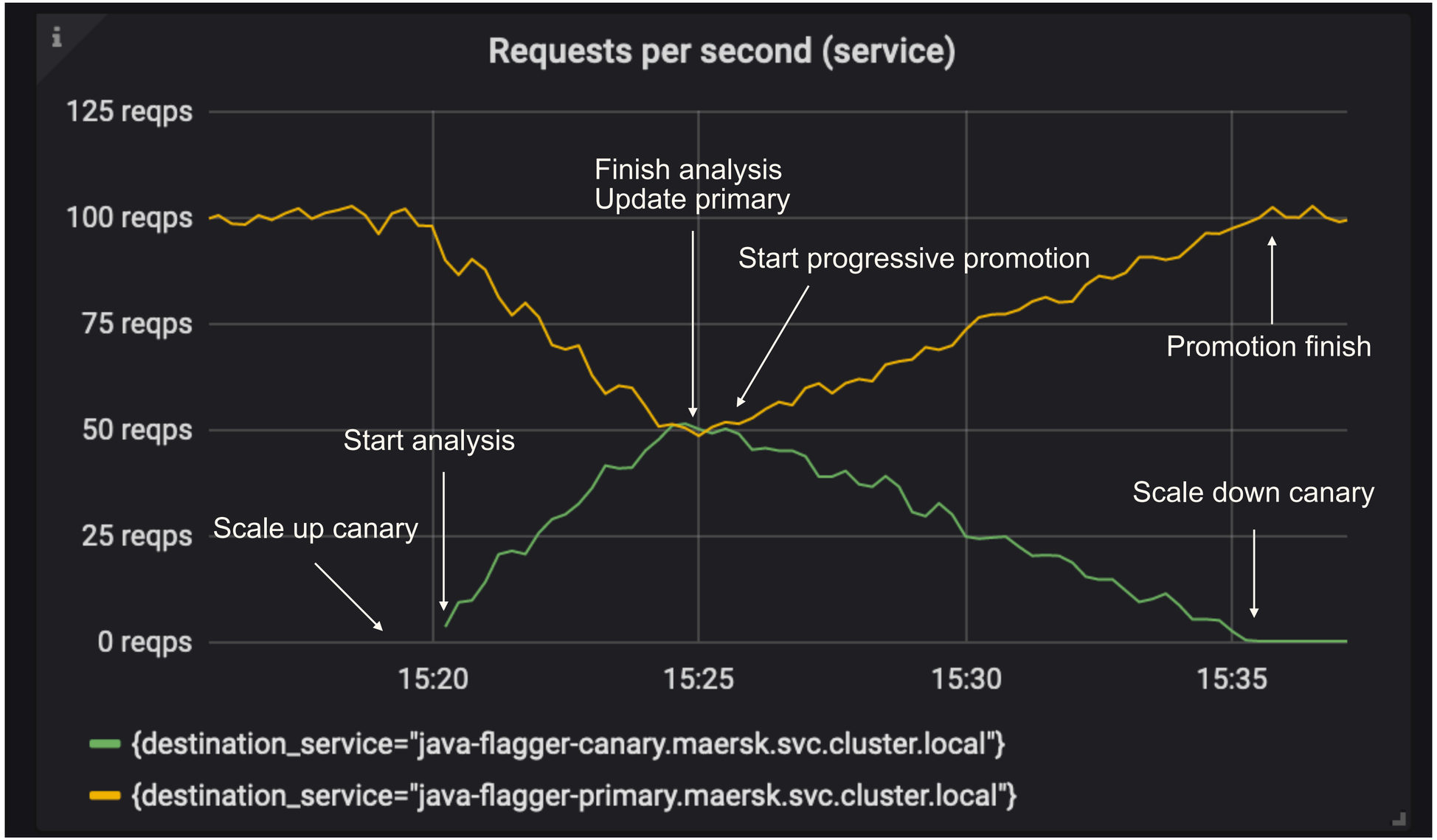
- 初期化後targetに変更が入るのを待つ
  - Target = deployment, daemonset
    - spec.templateのハッシュ値の変化
  - Targetがマウントしてるconfigmap, secretの変化も検知
- 検知した時点ではprimaryには反映されない(当たり前)

Promotion Process

- **canaryの rollout: Replicas = 0 を削除**
  - Healthyにならなければrollback
- **100%のトラフィックがcanaryに流れるまで、以下を繰り返す**
  - 数%のトラフィックをcanaryにshiftingする
  - metricsを確認: 指定されたrangeの中に入っているか
    - 入っていなければrollback
- **canaryからprimaryへdeployment, configmap, secret, hpaをコピーする**
  - User managedなresourceをFlagger managedなresourceへコピー
- **トラフィックをprimaryへ戻す**
  - 段階的に戻すことも可能 (progressive promotion)

※あくまで一例

Promotion Process



https://github.com/weaveworks/flagger/pull/593

Rollback

- rollback = 元のdeployment, daemonsetに紐づくpodを0個にする
  - deploymentの場合: ‘replicas: 0’ をセットする
  - daemonsetの場合: 存在しないlabelでnodeSelectorをセットする
- マニフェスト自体がロールバックされるわけではない
  - gitopsフレンドリー

Webhooks

- 各フェーズでwebhookを設定できる(2xx以外が返ってきたらrollback)
  - confirm-rollout … canaryのrollout前
  - pre-rollout … analysisを開始する前
  - rollout … analysisのループ毎
  - confirm-promotion … primaryにpromotionする前
  - post-rollout … primaryへのpromotion後

Considerations / Limitations

- podの数は単純に二倍
- serviceがflaggerによって作られる
  - 既存のserviceをそのまま使えない(使えるが、gitops的にはアウト)
- nativeのserviceを使ったcanary releaseができない(argo-rollouts方式)
  - Blue Greenは可能だが、基本的にはservice mesh, ingress controller前提
- HPAへの変更は検知されない
- 完全に自動化されるわけではない(絶対安全ではない)
  - metricsで確認できないバグやエラーはあり得る

Conclusion

- Progressive DeliveryはCD++
  - Productionリリースのriskを減らす
  - AgilityとReliabilityの両立
- Flaggerはk8s(service mesh)上でのProgressive Deliveryを実現
  - 複数のdeployment strategy, metrics provider,
  - configmap/secretの変更検知
- We welcome all contributions!
  - Stack driver対応, Statefulset対応, HPAの変更検知, etc.
- 質問等は気軽にweaveworks community slackにて: https://slack.weave.works/ #flagger

References

- https://github.com/weaveworks/flagger
- https://docs.flagger.app/
- https://www.weave.works/blog/announcing-flagger-1-0
- https://medium.com/google-cloud-jp/gke-istio-flagger%E3%81%AB%E3%82%88%E3%82%8Bprogressive-delivery-5f1ea9b627c1
- https://www.slideshare.net/weaveworks/whats-new-in-flagger-10-with-stefan-prodan
- https://medium.com/google-cloud/automated-canary-deployments-with-flagger-and-istio-ac747827f9d1
- https://medium.com/@dlorenc/pitfalls-of-progressive-delivery-114c6e3f9dbb
- https://carlossg.github.io/presentations/2019-06_cdsummit
- https://medium.com/@copyconstruct/testing-in-production-the-safe-way-18ca102d0ef1
- https://www.infoq.com/presentations/progressive-delivery
- https://qiita.com/mumoshu/items/63b29bca6a052d8c7087
- https://www.getambassador.io/docs/latest/topics/concepts/progressive-delivery/